

RAG-Augmented Multi-Agent LLM Framework for Explainable Software Vulnerability Detection Using CWE Knowledge Bases

Richa Verma (25MCSS02) Advisor: Dr. Akshay Pandey

A small book of CWE facts

Where this fits

We already have 12 short Java files and a pattern checker that matches them. That checker can raise an alarm. It cannot yet say the official name of the weakness or how to fix it.

This note describes the book we store those facts in. It does not change the detection score.

The idea we started with

An explanation that the tool invents cannot be audited. The words for the weakness, the risk, and the fix should be stored facts that a reader can open and compare.

We treated the CWE book as a source of truth for names, not as a detector. The book does not look at Java. Something else has to decide which page to open.

Why we chose this way

We did not load the entire MITRE catalog. Most pages are unrelated to the six weaknesses, and a huge book makes search noisier. A subset is honest if we say it is a subset.

We kept parent and child links, not only the six exact ids. Public suites often use a nearby id, such as CWE-80 for a kind of cross-site scripting when our list says CWE-79. If the book has no neighbor pages, a correct nearby name looks like a hallucination.

Fixes are stored beside the description so the answer can suggest a repair. The book has no field for how to attack the code. That was a deliberate omission.

What we actually did

CWE pages are published by MITRE. We do not copy the whole catalog. We keep the pages we need for the six weaknesses, plus some close neighbors, so a near miss can still be named.

Each stored page has an id, a name, a short description, a longer description when MITRE provides one, links to parent and child weaknesses, and suggested fixes.

The tool can look up one id, search by words such as "cross site scripting", and list neighbors. The pattern checker is not allowed to invent this text. If the answer names a CWE, that name has to come from this book.

Take CWE-89 as the example in the picture. The page says SQL injection. It says what goes wrong when user text is joined into a query. It also points at related pages and at a safer style of query.

A parent weakness is a broader group. A child weakness is a more specific case. We keep both so the explanation can stay honest when the code is close but not exact.

A concrete example

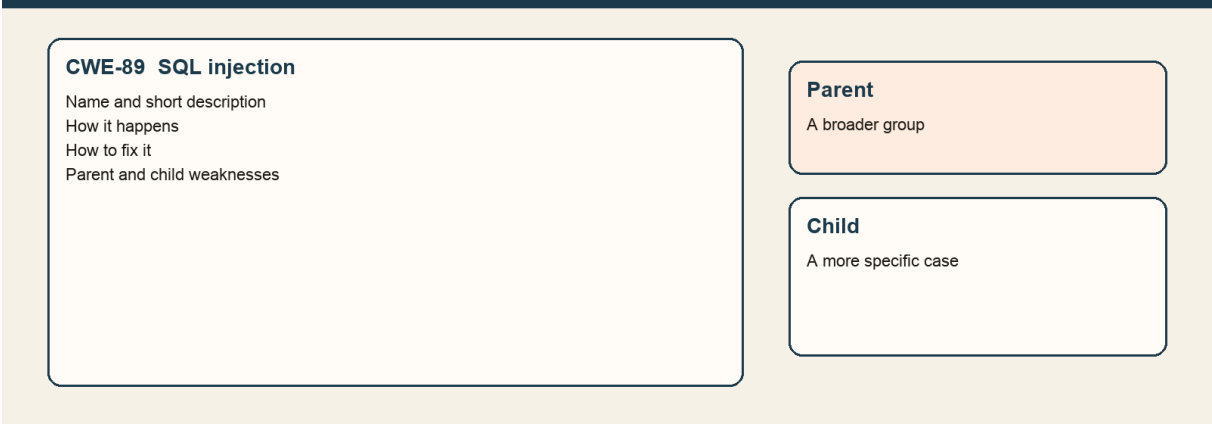
Ask the book for CWE-89. It returns the name SQL injection, a short description, and a fix such

as keeping user text out of the query command.

Ask it for the neighbors of CWE-89. Parents are broader groups. Children are more specific kinds of injection. The answer can say "this is SQL injection, which sits under this broader group" without inventing a new name.

Search the words "cross site scripting". The top hit should be the XSS page, CWE-79, not the SQL page. That is a lookup test, not a test of whether some Java file is unsafe.

Picture



One stored page, CWE-89, and the kind of related pages we keep beside it.

The numbers

On each page	Why we keep it
Id and name	So the answer uses the official name.
Description	So the note can say what the weakness is.
Parents and children	So a close match can still be explained.
Fixes	So the answer can suggest a repair, not an attack.

This book is a subset. The full CWE list is much larger. We record that limit instead of pretending the book is complete.

What we concluded, and why

The book can be queried by a person. The Java file does not search it by itself yet.

The next piece of work is to turn the risky part of a Java file into a question and find the best CWE pages for that question. The conclusion of this note is that names and fixes now have a source. Detection is unchanged.

What we are not saying

Storing the book does not make detection better. The score on the 12 practice files is unchanged.